

Project Documentation: Moore Rising Edge Detector Unit

Digital Design Lab

Project Overview

This lab implements a **Moore Finite State Machine (FSM)** for rising edge detection in Verilog HDL[cite: 33]. The circuit monitors an input signal (`level`) and asserts a single clock-cycle pulse (`tick`) whenever a $0 \rightarrow 1$ transition is detected[cite: 33]. Because it is designed as a Moore FSM, the output `tick` depends strictly on the current state (`EDGE`) rather than directly on combinational inputs[cite: 33].

Specifications & Functionality

- **States:**
 - **ZERO** (`2'b00`): Idle state indicating a low input level[cite: 33]. Output `tick` = 0[cite: 33]. Transitions to **EDGE** if `level` = 1[cite: 33].
 - **EDGE** (`2'b01`): Edge detection state[cite: 33]. Asserts output `tick` = 1[cite: 33]. Transitions to **ONE** if `level` = 1, or back to **ZERO** if `level` = 0[cite: 33].
 - **ONE** (`2'b10`): High level state[cite: 33]. Output `tick` = 0[cite: 33]. Remains in **ONE** while `level` = 1; returns to **ZERO** when `level` = 0[cite: 33].
- **Asynchronous Reset:** Active-high reset immediately forces state register to **ZERO** on `posedge reset`[cite: 33].
- **Moore Output Logic:** Output `tick` is driven high if and only if `current_state == EDGE`[cite: 33].

Port Summary

Port Name	Direction	Type	Width	Description
<code>clk</code>	Input	<code>wire</code>	1	Clock input signal[cite: 33, 34]
<code>reset</code>	Input	<code>wire</code>	1	Asynchronous active-high reset[cite: 33, 34]
<code>level</code>	Input	<code>wire</code>	1	Input level signal to monitor[cite: 33, 34]
<code>tick</code>	Output	<code>reg</code>	1	Single clock cycle pulse output[cite: 33, 34]

Table 1: Rising Edge Detector Port Interfaces

Verilog Source Code

Design Module (`rising_edge_moore.v`)

```
module rising_edge_moore (
    input wire clk,
    input wire reset,
    input wire level,
    output reg tick
);

parameter ZERO = 2'b00;
parameter EDGE = 2'b01;
parameter ONE  = 2'b10;

reg [1:0] current_state, next_state;

// State Register with Asynchronous Reset
always @(posedge clk or posedge reset) begin
    if (reset)
        current_state <= ZERO;
    else
        current_state <= next_state;
end

// Next State Logic
always @(*) begin
    case (current_state)
        ZERO: begin
            if (level)
                next_state = EDGE;
            else
                next_state = ZERO;
        end
        EDGE: begin
            if (level)
                next_state = ONE;
            else
                next_state = ZERO;
        end
        ONE: begin
            if (level)
                next_state = ONE;
            else
                next_state = ZERO;
        end
        default:
            next_state = ZERO;
    endcase
end

// Moore Output Logic
always @(*) begin
    if (current_state == EDGE)
        tick = 1'b1;
    else
        tick = 1'b0;
end
end
```

```
endmodule
```

Testbench Module (tb_rising_edge_moore.v)

```
'timescale 1ns / 1ps

module tb_rising_edge_moore;

    reg clk;
    reg reset;
    reg level;

    wire tick;

    rising_edge_moore DUT (
        .clk      (clk),
        .reset    (reset),
        .level    (level),
        .tick     (tick)
    );

    // Clock Generation (10ns Period)
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end

    // Stimulus Pattern
    initial begin
        reset = 1'b1;
        level = 1'b0;

        #10;
        reset = 1'b0;

        #10;
        level = 1'b1;

        #20;
        level = 1'b1;

        #20;
        level = 1'b0;

        #20;
        level = 1'b1;

        #20;
        level = 1'b0;

        #20;
        $finish;
    end

    // Monitor Output
```

```
initial begin
    $monitor(
        "Time = %0t | clk = %b | reset = %b | level = %b | tick = %b",
        $time,
        clk,
        reset,
        level,
        tick
    );
end

endmodule
```

Simulation & Verification

Execution Command

To execute batch simulation using Cadence Xcelium or ModelSim:

```
xrun rising_edge_moore.v tb_rising_edge_moore.v
```

Expected Simulation Output

The simulation verifies that `tick` goes high for exactly one clock cycle following each low-to-high transition on `level`[cite: 33, 34]:

```
Time = 0 | clk = 0 | reset = 1 | level = 0 | tick = 0
Time = 5 | clk = 1 | reset = 1 | level = 0 | tick = 0
Time = 10 | clk = 0 | reset = 0 | level = 0 | tick = 0
Time = 20 | clk = 0 | reset = 0 | level = 1 | tick = 0
Time = 25 | clk = 1 | reset = 0 | level = 1 | tick = 1
Time = 35 | clk = 1 | reset = 0 | level = 1 | tick = 0
Time = 60 | clk = 0 | reset = 0 | level = 0 | tick = 0
Time = 80 | clk = 0 | reset = 0 | level = 1 | tick = 0
Time = 85 | clk = 1 | reset = 0 | level = 1 | tick = 1
Time = 95 | clk = 1 | reset = 0 | level = 1 | tick = 0
```